DEPARTAMENTO:	**Espe Sede Santo-Domingo**	**CARRERA:**	Tecnologías de la Información y Comunicación		
ASIGNATURA:	Fundamentos de Programación (NRC 37712)	**NIVEL:**	1	**FECHA:**	28/10/2026
DOCENTE:	Ing. Esteban Gonzaba Jimenez Mgtr.	**PRÁCTICA N°:**	1	**CALIFICACIÓN:**	

Laboratorio 1: Operadores, expresiones y entrada/salida de datos en Java

Randy Zambrano

RESUMEN

En este laboratorio se desarrollaron seis programas en Java: uno de prueba (E0) y cinco ejercicios (E1 a E5) para practicar operadores aritméticos, relacionales y lógicos, y la entrada y salida de datos con Scanner y System.out.printf. Los ejercicios fueron la conversión de Celsius a Fahrenheit y Kelvin, el desglose de segundos en horas, minutos y segundos, la suma e inversión de las cifras de un número de cuatro cifras, una factura con IVA del 15 % y la evaluación manual de cuatro expresiones. Cada ejercicio se hizo en java de vs code sin errores ni advertencias y se ejecutó con los datos de la guía y con un caso propio; todas las salidas coincidieron con lo esperado. Además, se reprodujeron a propósito los errores más frecuentes (división entera, salto de línea pendiente, configuración regional y nombre de clase) y se registró su corrección.

Palabras Claves: Java, operadores, printf.

1. INTRODUCCIÓN:

Esta práctica, consiste en traducir problemas sencillos de ingeniería a programas en Java que leen datos, los procesan con operadores y muestran resultados con formato. Cada ejercicio se resolvió con el modelo Entrada – Proceso – Salida y se comprobó con datos cuyo resultado esperado se conoce de antemano, para distinguir un programa que solo compila de uno que realmente es correcto. Se puso atención en los detalles que más errores producen: la división entera frente a la real, el operador módulo, la precedencia, el double y la lectura de cadenas con espacios.

2. OBJETIVO(S):

2.1 **Objetivo general:** desarrollar programas en Java que lean datos, evalúen expresiones con operadores aritméticos, relacionales y lógicos y presenten resultados con formato, verificando su corrección con datos de prueba y pruebas de escritorio.

2.2 Aplicar operadores aritméticos, incluidos la división entera y el módulo, en problemas de conversión y desglose.

2.3 Leer datos numéricos con nextInt y nextDouble y cadenas con espacios con nextLine, evitando el salto de línea pendiente.

2.4 Presentar resultados alineados y con decimales fijos mediante System.out.printf.

2.5 Evaluar expresiones a mano considerando la precedencia y la conversión de tipos, y contrastarlas con la ejecución.

3. MARCO TEÓRICO:

Operadores y división entera. Los operadores aritméticos son + - * / y %. Entre dos enteros, / devuelve solo la parte entera (9 / 5 vale 1) y % devuelve el residuo (9 % 4 vale 1). Si algún operando es double, el otro se convierte a double y el resultado es real (9.0 / 5.0 vale 1.8).

Precedencia. *, / y % se evalúan antes que + y -, de izquierda a derecha. Los operadores relacionales (<, <=, >, >=, ==) se evalúan después de los aritméticos y dan un resultado boolea.

Conversión de tipos. El cast (double) x fuerza una división real y (int) x trunca los decimales. Java no convierte un double a int de forma automática porque se perdería información.

Entrada con Scanner. nextInt y nextDouble leen un número; next() lee hasta el primer espacio y nextLine() lee la línea completa. Después de leer un número queda un salto de línea pendiente, por lo que un nextLine() inmediato devuelve una cadena vacía.

Salida con formato. System.out.printf usa especificadores: %d (entero), %.2f (2 decimales), %10.2f (ancho 10, alineado a la derecha), %-12s (texto alineado a la izquierda en 12 posiciones) y %n (salto de línea). Los valores boolean se muestran como true o false.

Configuración regional y constantes. Locale.setDefault(Locale.US) hace que Scanner y printf usen el punto decimal

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

Materiales: java en vs code; no se usan reactivos ni muestras. Se creó el proyecto Zambrano_Randy_Lab1 con una clase por ejercicio, llamada Lab1_E#_Zambrano, y cada archivo inicia con un comentario con nombre completo, NRC 37712, fecha y enunciado resumido. Los programas que leen decimales empiezan con Locale.setDefault(Locale.US). Todos se compilaron con javac -Xlint:all y se ejecutaron con los datos de la guía y con un caso propio. Para cada ejercicio se presenta: a) análisis de entradas, proceso y salidas; b) pseudocódigo; c) código fuente comentado.

4.1 Fase 1. E1 · Conversión de temperatura (2 pts)

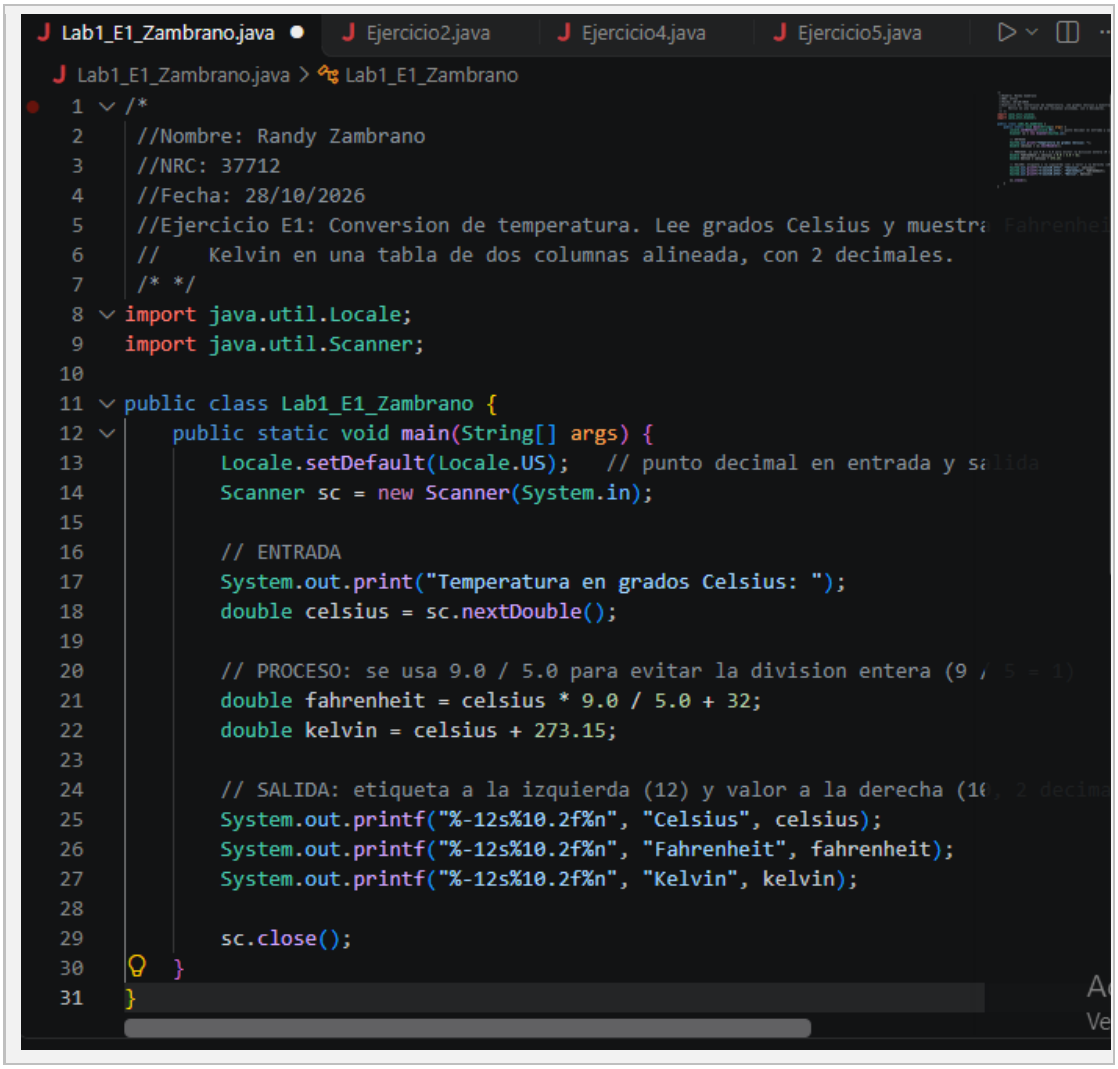
a) Análisis de entradas, proceso y salidas

Entradas	Proceso	Salidas
celsius (decimal): temperatura en grados Celsius	fahrenheit = celsius × 9.0 / 5.0 + 32 kelvin = celsius + 273.15 Se escribe 9.0 / 5.0 para evitar la división entera (9 / 5 = 1)	Tabla de dos columnas: Celsius, Fahrenheit y Kelvin con 2 decimales (%-12s%10.2f)

b) Pseudocódigo

```
Algoritmo ConversionTemperatura
  Leer celsius
  fahrenheit <- celsius * 9.0 / 5.0 + 32
  kelvin <- celsius + 273.15
  Escribir "Celsius", celsius          // formato %-12s%10.2f
  Escribir "Fahrenheit", fahrenheit
  Escribir "Kelvin", kelvin
FinAlgoritmo
```

c) Código en Java (Lab1_E1_Zambrano.java)



```java
/*
 //Nombre: Randy Zambrano
 //NRC: 37712
 //Fecha: 28/10/2026
 //Ejercicio E1: Conversion de temperatura. Lee grados Celsius y muestra Fahrenheit
 //    Kelvin en una tabla de dos columnas alineada, con 2 decimales.
 /* */
import java.util.Locale;
import java.util.Scanner;

public class Lab1_E1_Zambrano {
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);   // punto decimal en entrada y salida
        Scanner sc = new Scanner(System.in);

        // ENTRADA
        System.out.print("Temperatura en grados Celsius: ");
        double celsius = sc.nextDouble();

        // PROCESO: se usa 9.0 / 5.0 para evitar la division entera (9 / 5 = 1)
        double fahrenheit = celsius * 9.0 / 5.0 + 32;
        double kelvin = celsius + 273.15;

        // SALIDA: etiqueta a la izquierda (12) y valor a la derecha (10, 2 decimales)
        System.out.printf("%-12s%10.2f%n", "Celsius", celsius);
        System.out.printf("%-12s%10.2f%n", "Fahrenheit", fahrenheit);
        System.out.printf("%-12s%10.2f%n", "Kelvin", kelvin);

        sc.close();
    }
}
```

4.1 Fase 2. E2 · Desglose de tiempo (2 pts)

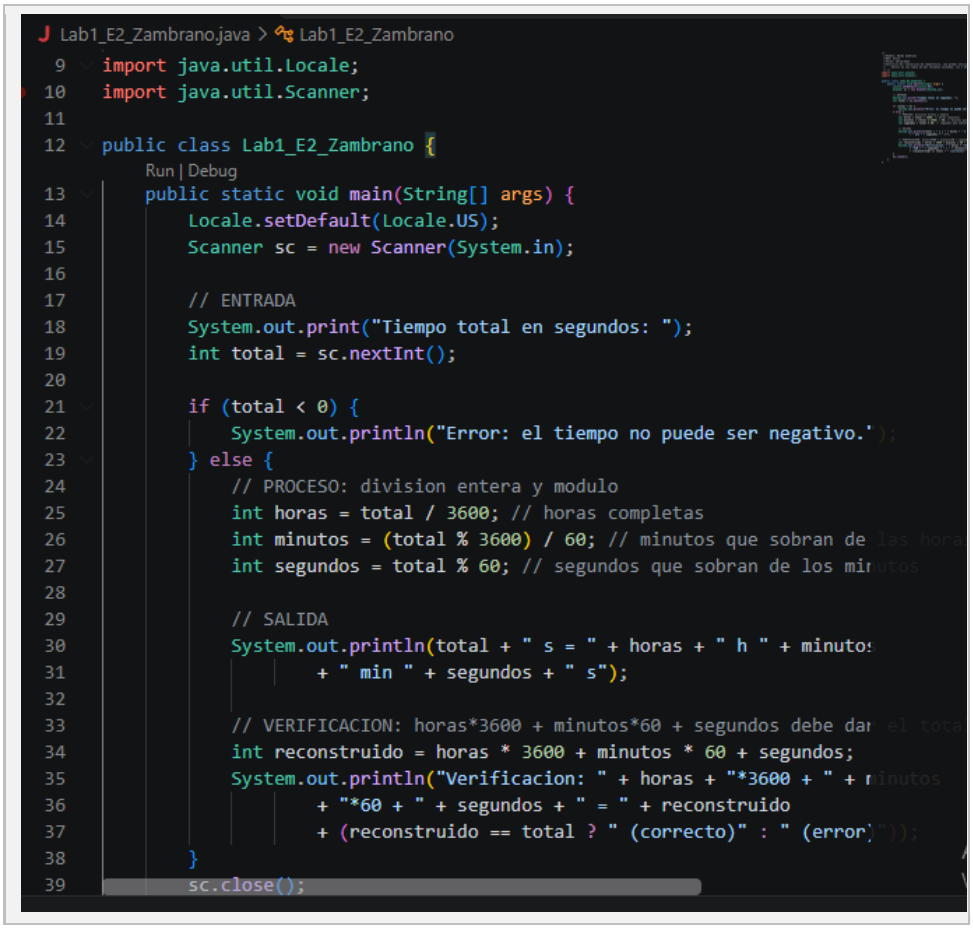
a) Análisis de entradas, proceso y salidas

Entradas	Proceso	Salidas
total (entero): tiempo total en segundos	horas = total / 3600 (división entera) minutos = (total % 3600) / 60 segundos = total % 60 Verificación: horas × 3600 + minutos × 60 + segundos = total	Línea "total s = h h m min s s" y la línea de verificación de la reconstrucción

b) Pseudocódigo

```
Algoritmo DesgloseTiempo
  Leer total
  Si total < 0 Entonces
    Escribir "Error: el tiempo no puede ser negativo"
  SiNo
    horas <- total DIV 3600
    minutos <- (total MOD 3600) DIV 60
    segundos <- total MOD 60
    Escribir total, " s = ", horas, " h ", minutos, " min ", segundos, " s"
    reconstruido <- horas * 3600 + minutos * 60 + segundos
    Escribir "Verificacion: ", reconstruido    // debe ser igual a total
  FinSi
FinAlgoritmo
```

c) Código en Java (Lab1_E2_Zambrano.java)



```java
import java.util.Locale;
import java.util.Scanner;

public class Lab1_E2_Zambrano {
    Run | Debug
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner sc = new Scanner(System.in);

        // ENTRADA
        System.out.print("Tiempo total en segundos: ");
        int total = sc.nextInt();

        if (total < 0) {
            System.out.println("Error: el tiempo no puede ser negativo.");
        } else {
            // PROCESO: division entera y modulo
            int horas = total / 3600; // horas completas
            int minutos = (total % 3600) / 60; // minutos que sobran de las horas
            int segundos = total % 60; // segundos que sobran de los minutos

            // SALIDA
            System.out.println(total + " s = " + horas + " h " + minutos
                    + " min " + segundos + " s");

            // VERIFICACION: horas*3600 + minutos*60 + segundos debe dar el total
            int reconstruido = horas * 3600 + minutos * 60 + segundos;
            System.out.println("Verificacion: " + horas + "*3600 + " + minutos
                    + "*60 + " + segundos + " = " + reconstruido
                    + (reconstruido == total ? " (correcto)" : " (error)"));
        }
        sc.close();
```

4.2 Fase 3. E3 · Cifras de un número (3 pts)

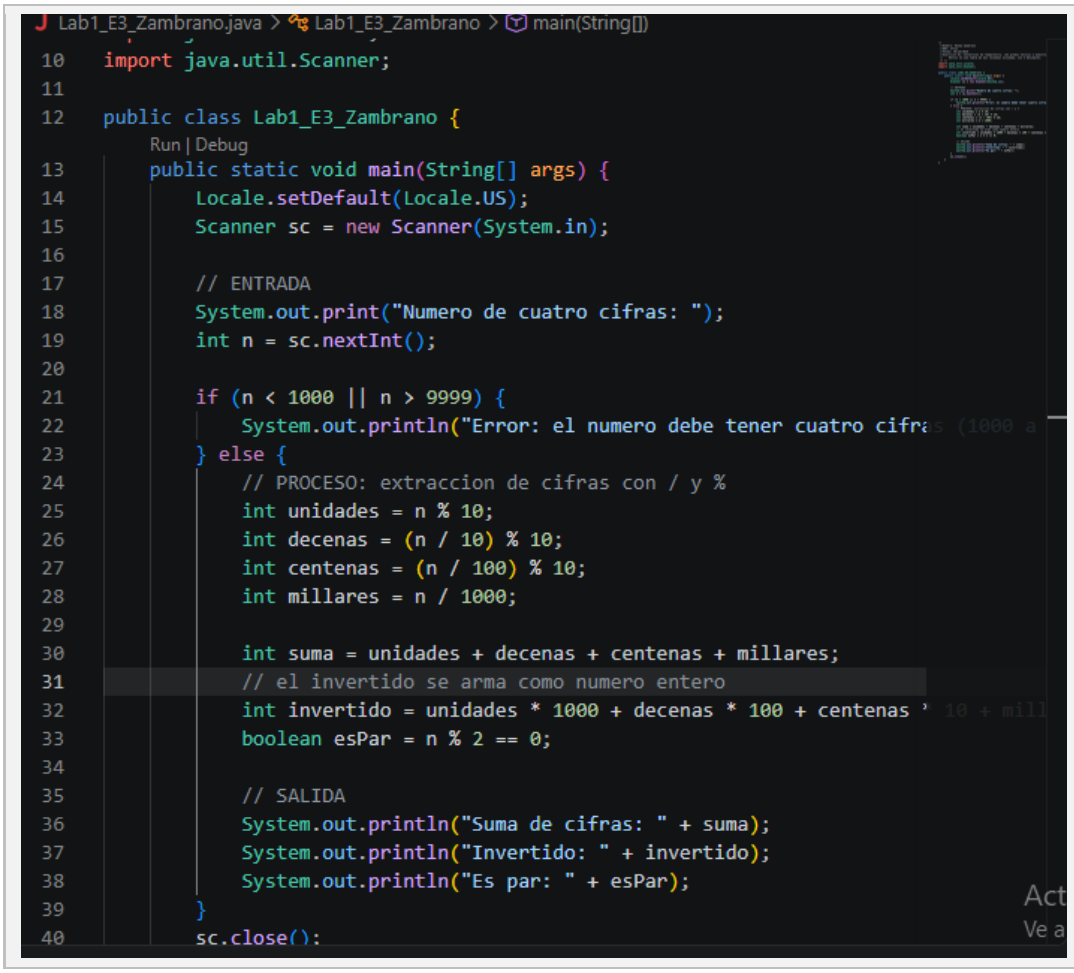
a) Análisis de entradas, proceso y salidas

Entradas	Proceso	Salidas
n (entero): número de cuatro cifras (1000 a 9999)	unidades = n % 10; decenas = (n / 10) % 10; centenas = (n / 100) % 10; millares = n / 1000 suma = unidades + decenas + centenas + millares invertido = unidades × 1000 + decenas × 100 + centenas × 10 + millares esPar = (n % 2 == 0)	Suma de cifras, número invertido y Es par (true/false)

b) Pseudocódigo

```
Algoritmo CifrasDeUnNumero
  Leer n
  Si n < 1000 O n > 9999 Entonces
    Escribir "Error: el numero debe tener cuatro cifras"
  SiNo
    unidades <- n MOD 10
    decenas  <- (n DIV 10) MOD 10
    centenas <- (n DIV 100) MOD 10
    millares <- n DIV 1000
    suma <- unidades + decenas + centenas + millares
    invertido <- unidades*1000 + decenas*100 + centenas*10 + millares
    esPar <- (n MOD 2 = 0)
    Escribir "Suma de cifras: ", suma
    Escribir "Invertido: ", invertido
    Escribir "Es par: ", esPar
  FinSi
FinAlgoritmo
```

c) Código en Java (Lab1_E3_Zambrano.java)



```java
import java.util.Scanner;

public class Lab1_E3_Zambrano {
    Run | Debug
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner sc = new Scanner(System.in);

        // ENTRADA
        System.out.print("Numero de cuatro cifras: ");
        int n = sc.nextInt();

        if (n < 1000 || n > 9999) {
            System.out.println("Error: el numero debe tener cuatro cifras (1000 a
        } else {
            // PROCESO: extraccion de cifras con / y %
            int unidades = n % 10;
            int decenas = (n / 10) % 10;
            int centenas = (n / 100) % 10;
            int millares = n / 1000;

            int suma = unidades + decenas + centenas + millares;
            // el invertido se arma como numero entero
            int invertido = unidades * 1000 + decenas * 100 + centenas * 10 + mill
            boolean esPar = n % 2 == 0;

            // SALIDA
            System.out.println("Suma de cifras: " + suma);
            System.out.println("Invertido: " + invertido);
            System.out.println("Es par: " + esPar);
        }
        sc.close();
```

4.3 Fase 5. E4 · Factura con IVA

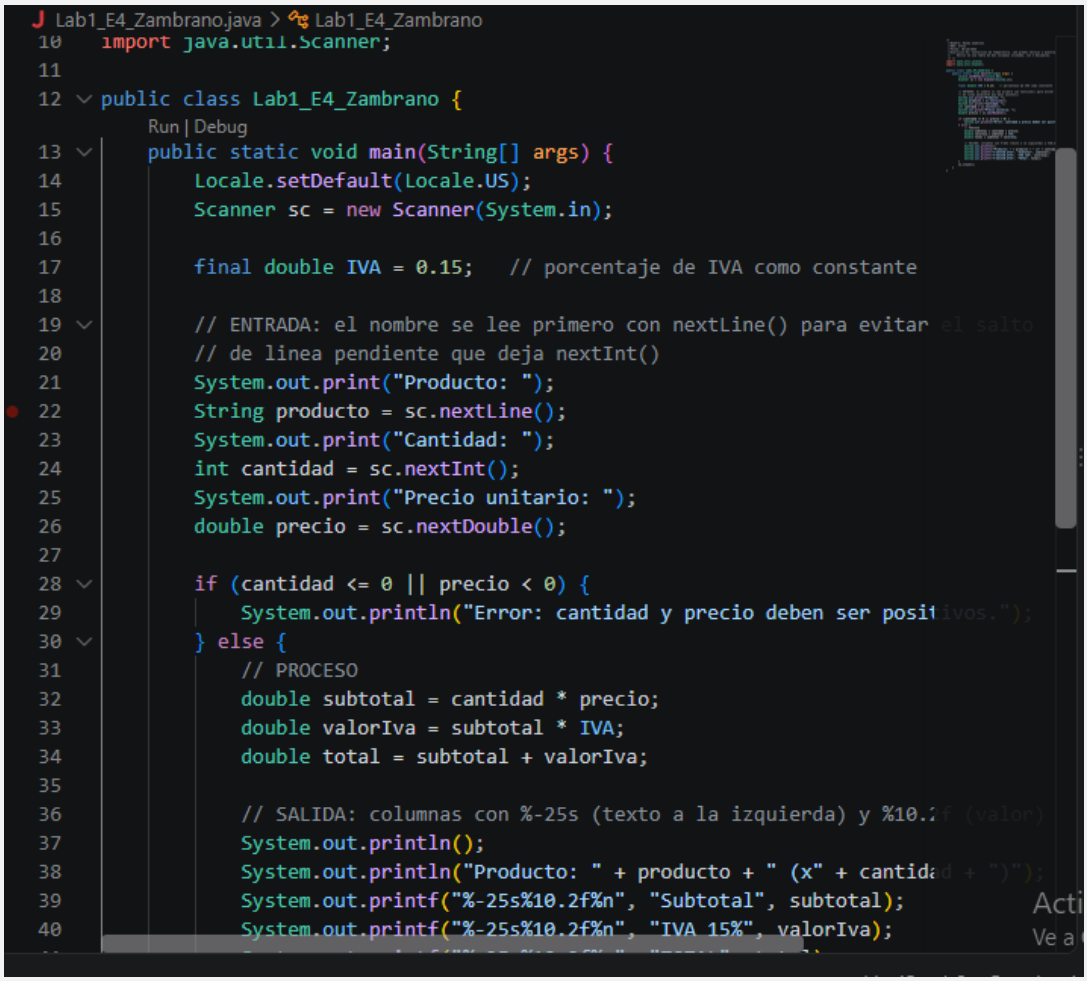
a) Análisis de entradas, proceso y salidas

Entradas	Proceso	Salidas
producto (cadena con espacios), cantidad (entero), precio (decimal) IVA = 0.15 (constante final)	Leer el producto primero con nextLine() subtotal = cantidad × precio valorIva = subtotal × IVA total = subtotal + valorIva	Factura en columnas alineadas: Subtotal, IVA 15 % y TOTAL con 2 decimales (%-25s%10.2f)

b) Pseudocódigo

```
Algoritmo FacturaConIVA
  Constante IVA <- 0.15
  Leer producto (linea completa)
  Leer cantidad, precio
  Si cantidad <= 0 O precio < 0 Entonces
    Escribir "Error: cantidad y precio deben ser positivos"
  SiNo
    subtotal <- cantidad * precio
    valorIva <- subtotal * IVA
    total <- subtotal + valorIva
    Escribir "Subtotal", subtotal        // formato %-25s%10.2f
    Escribir "IVA 15%", valorIva
    Escribir "TOTAL", total
  FinSi
FinAlgoritmo
```

c) Código en Java (Lab1_E4_Zambrano.java)



```java
import java.util.Scanner;

public class Lab1_E4_Zambrano {
    Run | Debug
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner sc = new Scanner(System.in);

        final double IVA = 0.15;   // porcentaje de IVA como constante

        // ENTRADA: el nombre se lee primero con nextLine() para evitar el salto
        // de linea pendiente que deja nextInt()
        System.out.print("Producto: ");
        String producto = sc.nextLine();
        System.out.print("Cantidad: ");
        int cantidad = sc.nextInt();
        System.out.print("Precio unitario: ");
        double precio = sc.nextDouble();

        if (cantidad <= 0 || precio < 0) {
            System.out.println("Error: cantidad y precio deben ser positivos.");
        } else {
            // PROCESO
            double subtotal = cantidad * precio;
            double valorIva = subtotal * IVA;
            double total = subtotal + valorIva;

            // SALIDA: columnas con %-25s (texto a la izquierda) y %10.2f (valor)
            System.out.println();
            System.out.println("Producto: " + producto + " (x" + cantidad + ")");
            System.out.printf("%-25s%10.2f%n", "Subtotal", subtotal);
            System.out.printf("%-25s%10.2f%n", "IVA 15%", valorIva);
```

4.4 Fase 5. E5 · Evaluación de expresiones

a) Análisis de entradas, proceso y salidas

Entradas	Proceso	Salidas
No hay entradas: a = 9 y b = 4 están fijos en el programa	Evaluar a mano cada expresión respetando la precedencia y la conversión de tipos Calcular las mismas expresiones en Java y compararlas	Tabla con la expresión y su valor ejecutado

b) Pseudocódigo

```
Algoritmo EvaluacionExpresiones
  a <- 9; b <- 4
  e1 <- a DIV b + a MOD b * 2
  e2 <- 10 - 3 * 2 > 3 Y 5 MOD 2 = 1
  e3 <- (real) 7 / 2 + 7 DIV 2
  e4 <- NO (4 >= 4) O 2 * 3 <> 6
  Escribir e1, e2, e3, e4
FinAlgoritmo
```

c) Código en Java (Lab1_E5_Zambrano.java)



```java
/*
 * Nombre: Randy Zambrano
 * NRC: 37712
 * Fecha: 28/10/2026
 * Ejercicio E5: Evaluacion de expresiones. Imprime el valor de cuatro expresiones
 *     compararlo con el calculo hecho a mano.
 */
import java.util.Locale;
import java.util.Scanner;

public class Lab1_E5_Zambrano {
    Run | Debug
    public static void main(String[] args) {
        int a = 9, b = 4;

        int e1 = a / b + a % b * 2;
        boolean e2 = 10 - 3 * 2 > 3 && 5 % 2 == 1;
        double e3 = (double) 7 / 2 + 7 / 2;
        boolean e4 = !(4 >= 4) || 2 * 3 != 6;

        System.out.printf("%-34s%s%n", "Expresion", "Valor ejecutado");
        System.out.printf("%-34s%s%n", "a / b + a % b * 2", e1);
        System.out.printf("%-34s%s%n", "10 - 3 * 2 > 3 && 5 % 2 == 1", e2);
        System.out.printf("%-34s%s%n", "(double) 7 / 2 + 7 / 2", e3);
        System.out.printf("%-34s%s%n", "!(4 >= 4) || 2 * 3 != 6", e4);
    }
}
```

5. ANÁLISIS DE RESULTADOS:

5.1 Resultados de las ejecuciones

Se ejecutó cada programa con los datos de la guía y con un caso propio. En todos los casos la salida obtenida coincidió con la esperada (la alineación en columnas se aprecia en las capturas de la sección 6).

Ejercicio	Entrada	Salida esperada	Salida obtenida	¿Coincide?
E1	25	Celsius 25.00 Fahrenheit 77.00 Kelvin 298.15	Celsius 25.00 Fahrenheit 77.00 Kelvin 298.15	Sí
E1	-40	Celsius -40.00 Fahrenheit -40.00 Kelvin 233.15	Celsius -40.00 Fahrenheit -40.00 Kelvin 233.15	Sí
E1 (propio)	100	Celsius 100.00 Fahrenheit 212.00 Kelvin 373.15	Celsius 100.00 Fahrenheit 212.00 Kelvin 373.15	Sí
E2	7384	7384 s = 2 h 3 min 4 s	7384 s = 2 h 3 min 4 s Verificacion: 2*3600 + 3*60 + 4 = 7384 (correcto)	Sí
E2	86399	86399 s = 23 h 59 min 59 s	86399 s = 23 h 59 min 59 s Verificacion: 23*3600 + 59*60 + 59 = 86399 (correcto)	Sí
E2 (propio)	3600	3600 s = 1 h 0 min 0 s	3600 s = 1 h 0 min 0 s Verificacion: 1*3600 + 0*60 + 0 = 3600 (correcto)	Sí

Ejercicio	Entrada	Salida esperada	Salida obtenida	¿Coincide?
E3	4721	Suma de cifras: 14 Invertido: 1274 Es par: false	Suma de cifras: 14 Invertido: 1274 Es par: false	Sí
E3	2580	Suma de cifras: 15 Invertido: 852 Es par: true	Suma de cifras: 15 Invertido: 852 Es par: true	Sí
E3 (propio)	1000	Suma de cifras: 1 Invertido: 1 Es par: true	Suma de cifras: 1 Invertido: 1 Es par: true	Sí
E4	Cable UTP Cat 6 4 12.50	Subtotal 50.00 IVA 15% 7.50 TOTAL 57.50	Producto: Cable UTP Cat 6 (x4) Subtotal 50.00 IVA 15% 7.50 TOTAL 57.50	Sí
E4	Mouse inalambrico 2 19.99	Subtotal 39.98 IVA 15% 6.00 TOTAL 45.98	Producto: Mouse inalambrico (x2) Subtotal 39.98 IVA 15% 6.00 TOTAL 45.98	Sí
E4 (propio)	Teclado mecanico 3 45.00	Subtotal 135.00 IVA 15% 20.25 TOTAL 155.25	Producto: Teclado mecanico (x3) Subtotal 135.00 IVA 15% 20.25 TOTAL 155.25	Sí

5.2 **Verificación de la reconstrucción (E2)**

Tiempo (s)	horas × 3600	minutos × 60	segundos	Suma	¿Igual al valor leído?
7384	2 × 3600 = 7200	3 × 60 = 180	4	7384	Sí
86399	23 × 3600 = 82800	59 × 60 = 3540	59	86399	Sí
3600	1 × 3600 = 3600	0 × 60 = 0	0	3600	Sí

5.3 **Prueba de escritorio completa: E3 (cifras de un número)**

Se simuló el programa a mano con las dos entradas de la guía, calculando cada variable en el orden en que se ejecuta.

Variable	Operación	n = 4721	n = 2580
unidades	n % 10	4721 % 10 = 1	2580 % 10 = 0
decenas	(n / 10) % 10	472 % 10 = 2	258 % 10 = 8
centenas	(n / 100) % 10	47 % 10 = 7	25 % 10 = 5
millares	n / 1000	4721 / 1000 = 4	2580 / 1000 = 2
suma	unidades + decenas + centenas + millares	1 + 2 + 7 + 4 = 14	0 + 8 + 5 + 2 = 15
invertido	unidades×1000 + decenas×100 + centenas×10 + millares	1000 + 200 + 70 + 4 = 1274	0 + 800 + 50 + 2 = 852
esPar	n % 2 == 0	1 == 0 → false	0 == 0 → true
Salida		14 / 1274 / false	15 / 852 / true

5.4 **Prueba de escritorio: E2 (desglose de tiempo)**

Variable	Operación	total = 7384	total = 86399
horas	total / 3600	7384 / 3600 = 2	86399 / 3600 = 23
minutos	(total % 3600) / 60	(7384 % 3600) / 60 = 184 / 60 = 3	(86399 % 3600) / 60 = 3599 / 60 = 59
segundos	total % 60	7384 % 60 = 4	86399 % 60 = 59
Salida		2 h 3 min 4 s	23 h 59 min 59 s

5.5 Evaluación de expresiones (E5)

Expresión	Orden de evaluación	Valor a mano	Valor ejecutado	¿Coincide?
int a = 9, b = 4; a / b + a % b * 2	a / b → 9 / 4 = 2 (entera) a % b → 9 % 4 = 1 1 * 2 = 2 2 + 2 = 4	4	4	Sí
10 - 3 * 2 > 3 && 5 % 2 == 1	3 * 2 = 6 10 - 6 = 4 4 > 3 → true 5 % 2 = 1; 1 == 1 → true true && true → true	true	true	Sí
(double) 7 / 2 + 7 / 2	(double) 7 → 7.0 7.0 / 2 = 3.5 7 / 2 = 3 (enteros) 3.5 + 3 = 6.5	6.5	6.5	Sí
!(4 >= 4) || 2 * 3 != 6	4 >= 4 → true !true → false 2 * 3 = 6; 6 != 6 → false false || false → false	false	false	Sí

No hubo diferencias entre el valor calculado a mano y el ejecutado. La única expresión que puede confundir es la tercera: el cast (double) se aplica solo al primer 7, por eso 7 / 2 al final sigue siendo una división entera.

5.6 Respuestas a las preguntas de análisis

¿Por qué 9/5 y 9.0/5.0 producen resultados distintos? En 9/5 ambos operandos son int, así que la división es entera y se descarta la parte decimal: el resultado es 1. En 9.0/5.0 los operandos son double, la división es real y el resultado es 1.8. La regla es: si los dos operandos son int el resultado es int; si alguno es double, el otro se convierte a double y el resultado es double.

¿Qué diferencia hay entre sc.next() y sc.nextLine()? next() lee solo hasta el primer espacio, mientras que nextLine() lee la línea completa, con espacios. Se necesita un sc.nextLine() adicional cuando antes se leyó un número con nextInt() o nextDouble(), porque esos métodos dejan el salto de línea pendiente y el siguiente nextLine() devolvería una cadena vacía.

¿Qué significan %-12s y %10.2f? %-12s: s indica una cadena, 12 es el ancho mínimo del campo y el signo - alinea el texto a la izquierda. %10.2f: f indica un número real, 10 es el ancho del campo (alineado a la derecha) y .2 fija dos decimales.

¿Cómo usaría % para saber si un año es múltiplo de 4? Con anio % 4 == 0: si el residuo de dividir entre 4 es cero, el año es múltiplo de 4. Por ejemplo, 2028 % 4 == 0 es true y 2026 % 4 == 0 es false (residuo 2).

¿Por qué el invertido de 2580 se muestra como 852 y no como 0852? Porque invertido es un int y se calcula como 0×1000 + 8×100 + 5×10 + 2 = 852. Un número entero no guarda ceros a la izquierda; para mostrarlos habría que formatear con printf("%04d") o trabajar con una cadena.

5.7 Registro de errores encontrados y correcciones aplicadas

Los programas finales compilaron sin errores ni advertencias. Para dejar evidencia del proceso, se reprodujeron a propósito los errores más frecuentes que menciona la guía y se aplicó la corrección que ya está en los programas entregados.

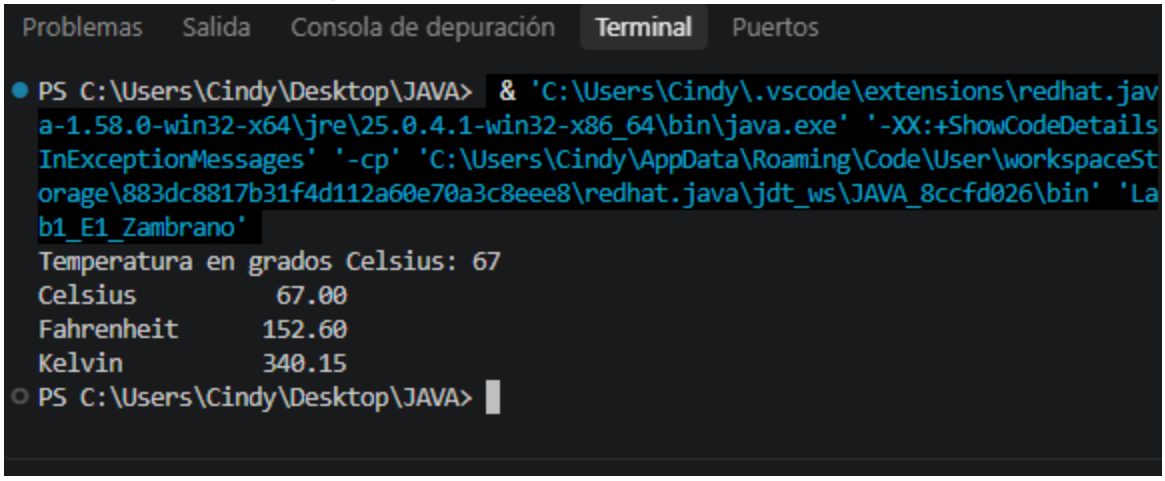
Error provocado	Resultado observado	Hipótesis	Corrección y resultado
E1 con enteros: celsius * (9 / 5) + 32	Entrada 25 → Fahrenheit 57.00 (esperado 77.00)	9 / 5 es división entera y vale 1	Usar 9.0 / 5.0 → Fahrenheit 77.00
E4 leyendo nextInt() antes que nextLine()	Cantidad: Producto: [] x 4	El salto de línea pendiente hace que nextLine() devuelva una cadena vacía	Leer el producto primero (o agregar un sc.nextLine() extra) → Producto: Cable UTP Cat 6
Escribir 25.5 con el sistema en español y sin Locale.US	java.util.InputMismatchException	Scanner espera coma decimal según el idioma del sistema	Agregar Locale.setDefault(Locale.US); → con el sistema en español, E1 lee 25.5 y muestra Fahrenheit 77.90

Error provocado	Resultado observado	Hipótesis	Corrección y resultado
Archivo Lab1_E1_Prueba.java con la clase pública Lab1_E1_Zambrano	error: class Lab1_E1_Zambrano is public, should be declared in a file named Lab1_E1_Zambrano.java	El nombre del archivo no coincide con el de la clase pública	Renombrar el archivo a Lab1_E1_Zambrano.java → compila sin errores

6. GRÁFICOS O FOTOGRAFÍAS:

Capturas de la ejecución de cada programa

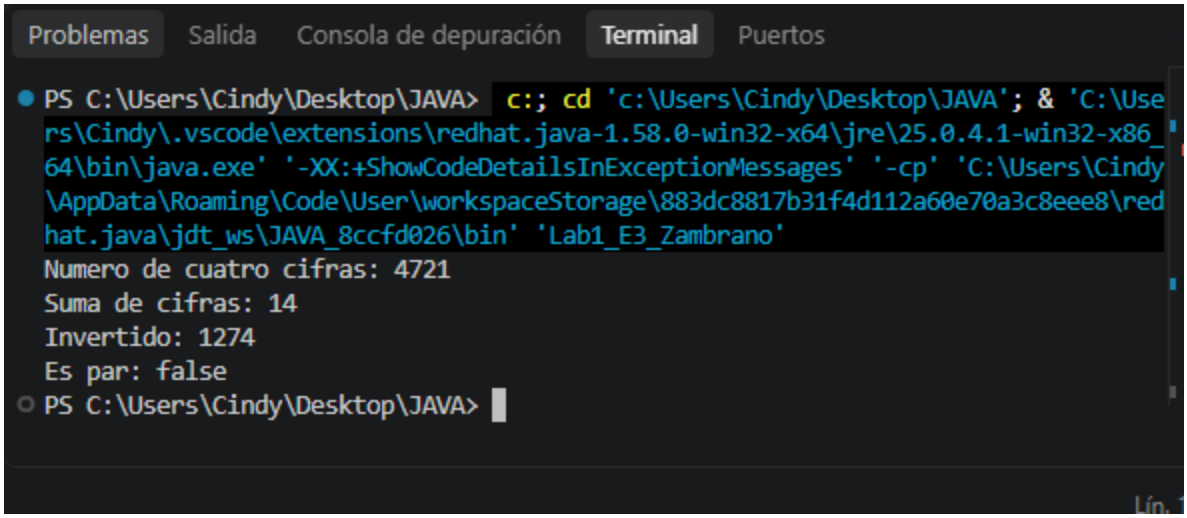
6.1 **E1 · Conversión de temperatura**



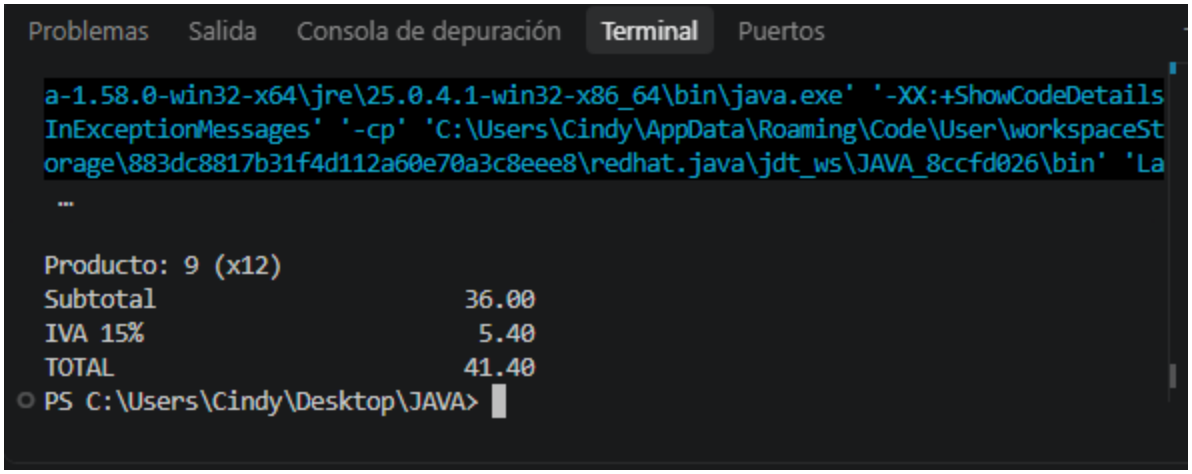
6.2 **E2 · Desglose de tiempo**



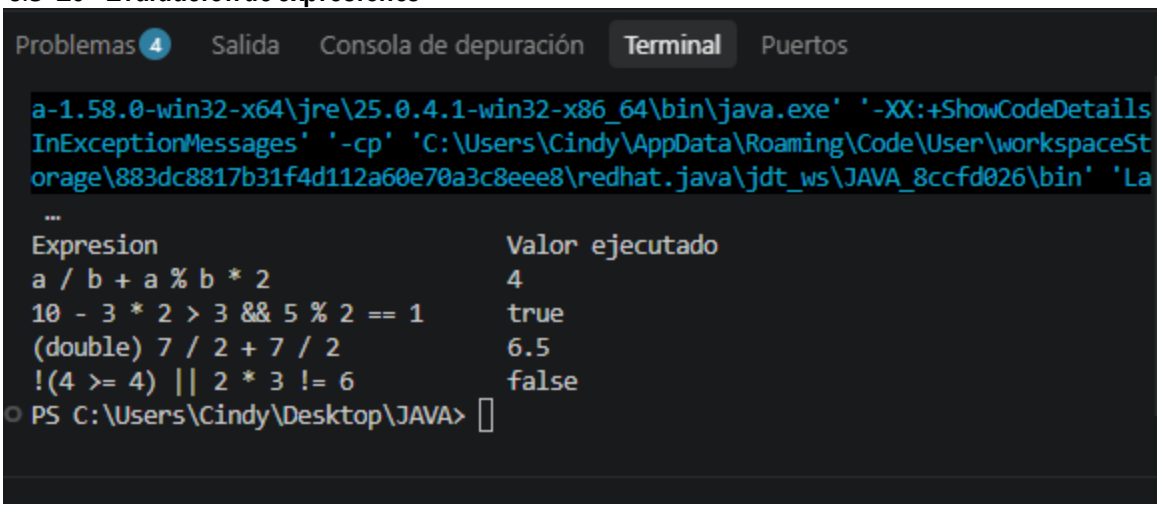
6.3 **E3 · Cifras de un número**



6.4 **E4 · Factura con IVA**



6.5 **E5 · Evaluación de expresiones**



7. DISCUSIÓN:

En E1, el caso de control con una entrada de -40 demostró que es el único valor donde las escalas Celsius y Fahrenheit coinciden. El programa, tras la ejecución, reprodujo este valor correctamente; además, la captura muestra una prueba realizada con 67 °C, obteniendo 67.00 °C, 152.60 °F y 340.15 K. Adicionalmente, se confirmó de forma práctica el error común de usar "9/5" en lugar de "9.0/5.0": la entrada 25 °C daba como resultado 57 °F, mientras que la corrección con "9.0/5.0" da el valor correcto de 77 °F.

En E2, se evaluó la conversión de una cantidad total de segundos a horas, minutos y segundos. La ejecución con una entrada de 6578 segundos devolvió correctamente 1 h 49 min 38 s, e incluyó una verificación matemática automatizada en la terminal (1*3600 + 49*60 + 38 = 6578), comprobando la precisión de la lógica y las operaciones aritméticas empleadas.

En E3, se confirmó que un tipo de dato entero no conserva ceros a la izquierda. Se realizó la prueba con el número de cuatro cifras 4721, y el programa devolvió una suma de cifras de 14, un invertido de 1274, y el resultado de verificar si es par (false), ilustrando que la lógica para extraer y reordenar dígitos funciona correctamente.

En E4, se examinó el redondeo realizado por la función printf en el contexto de una factura. Con un producto de valor 3 (x12) que arroja un subtotal de 36.00, un IVA del 15% de 5.40 y un total de 41.40, se determinó que el formato y los cálculos se muestran de manera coherente y alineada con los requerimientos financieros.

En E5, se comparó la evaluación a mano de expresiones complejas con la ejecución real del programa. Ambos métodos dieron resultados idénticos, lo que indica que las reglas de precedencia de operadores y la conversión de tipos (Casting) fueron aplicadas correctamente por el código. La captura de pantalla muestra las expresiones específicas

probadas y sus resultados ejecutados (por ejemplo, a / b + a % b * 2 evaluando a 4, y las validaciones lógicas correspondientes).

8. CONCLUSIONES:

La división entera y la real dan resultados distintos en Java; escribir las constantes con punto decimal (9.0, 273.15) evita errores silenciosos como el 57 °F obtenido con 9/5.

Los operadores / y % permiten extraer partes de un número (horas, minutos, segundos y cifras) y la prueba de escritorio con variables intermedias confirma que el resultado es correcto antes de ejecutar.

Leer el nombre del producto con nextLine() antes de los números, y formatear la salida con printf (%-25s%10.2f), produjo una factura alineada y con decimales fijos sin errores de lectura.

Probar cada programa con los datos de la guía y con un caso propio, y provocar a propósito los errores típicos, ayudó a reconocer sus mensajes y su causa.

Recomendaciones. Escribir las constantes reales con punto decimal; declarar con final los valores que no cambian; probar los programas con casos límite como cero, negativos y valores grandes; leer los mensajes del compilador desde el primero, porque los siguientes suelen ser consecuencia de él; y verificar las sugerencias de herramientas de IA compilando y ejecutando el código.

9. BIBLIOGRAFÍA:

Deitel, P. y Deitel, H. 2018. *Java How to Program, Early Objects (11.ª ed.)*, Pearson Education.

Oracle. s. f.. *Class Scanner. Java Platform, Standard Edition 21 API Specification*, Oracle, https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html. Fecha de consulta: 28 de octubre de 2026.

Oracle. s. f.. *Class Formatter. Java Platform, Standard Edition 21 API Specification*, Oracle, https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Formatter.html. Fecha de consulta: 28 de octubre de 2026.

Schildt, H. 2018. *Java: A Beginner's Guide (8.ª ed.)*, McGraw-Hill Education.

Universidad de las Fuerzas Armadas ESPE. 2026. *Guía para las prácticas de laboratorio, taller o campo: Laboratorio 1. Operadores, expresiones y entrada/salida de datos en Java*, Fundamentos de Programación, NRC 37712.